

高级语言程序设计大作业实验报告

南开大学 计算机大类

姓名：邵继昱 学号：2512303 班级：1010

2026 年 5 月 11 日

目录

1	作业题目	3
2	开发软件	3
3	课题要求	3
4	主要流程	3
4.1	整体流程	3
4.2	算法或公式	4
4.2.1	小鸟飞行	4
4.2.2	移动管道	5
4.2.3	皮肤切换	5
4.2.4	碰撞检测	6
4.2.5	分数计算	6
4.3	单元测试	6
5	单元测试	7
5.1	测试案例定义	7
5.2	测试结果	7
6	收获	7
6.1	虚析构函数	7
6.2	单元测试	7
6.3	面向对象设计	7
6.4	设计模式	8
6.5	Qt 框架	8

1 作业题目

基于 Qt6 的 Flappy Bird 游戏开发

2 开发软件

Visual Studio 2022 + Qt6

3 课题要求

- 面向对象程序设计。
- 设计模式应用。
- 完整项目实施。
- 单元测试。

4 主要流程

4.1 整体流程

实现思路：定义抽象的虚基类 `Pipe`，利用多态实现不同管道类型。
主要类继承关系如图 1 所示。

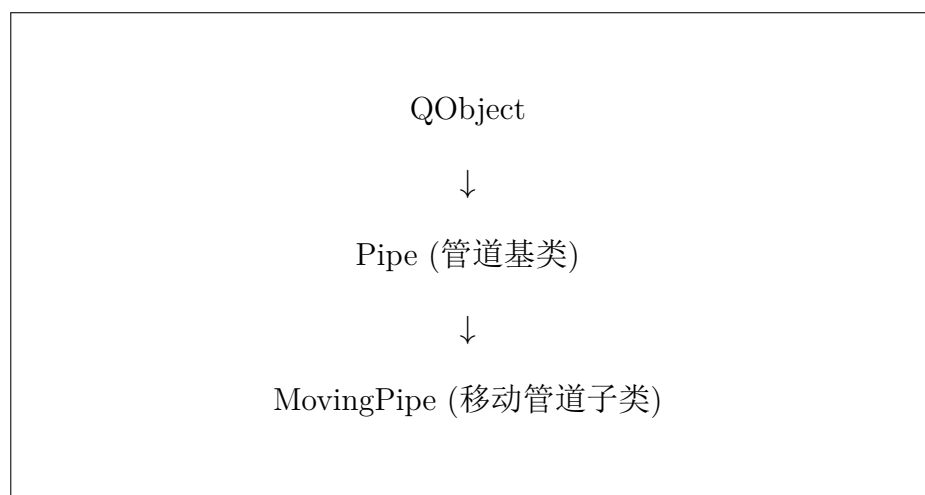


图 1: Pipe 类继承关系图

游戏整体流程如图 2 所示。

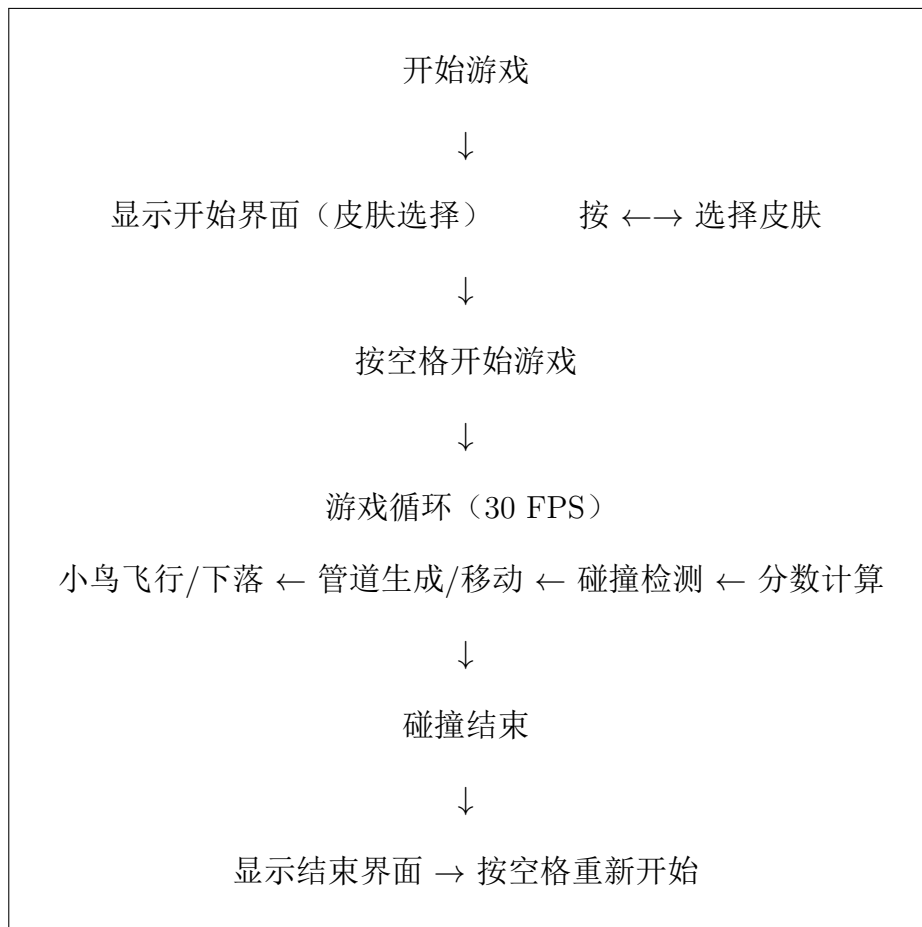


图 2: 游戏整体流程图

4.2 算法或公式

4.2.1 小鸟飞行

小鸟飞行物理模型：

- 飞行初速度： $v_{flap} = 14$
- 重力加速度： $a = 2$
- 最大下落速度： $v_{max} = 15$
- 速度更新： $v = v + a$ （每帧）
- 位置更新： $y = y - v$

```
1 void Bird::freeFall() {  
2     if (velocity < MAX_VEL_Y) {  
3         velocity -= ACC_Y; // 重力加速度  
4     }
```

```
5     y = qMin(y - velocity, BOTTOM_BOUNDARY); // 更新位置
6     collisionRect.translate(0, -velocity);    // 更新碰撞矩形
7 }
```

4.2.2 移动管道

移动管道算法：

- 移动方向：DIR_DOWN 向下，DIR_UP 向上
- 移动偏移量：dealtY
- 最大移动距离：MAX_DELTA

```
1 void MovingPipe::movement() {
2     // 水平移动（与普通管道相同）
3     x -= speed;
4     pipeRect.translate(-speed, 0);
5     if (x < -1 * PIPE_HEAD_WIDTH) {
6         visible = false;
7     }
8
9     // 垂直往复移动
10    if (direction == DIR_DOWN) {
11        dealtY++;
12        if (dealtY > MAX_DELTA) {
13            direction = DIR_UP;
14        }
15    } else {
16        dealtY--;
17        if (dealtY <= 0) {
18            direction = DIR_DOWN;
19        }
20    }
21    pipeRect.moveTo(x, y + dealtY);
22 }
```

4.2.3 皮肤切换

根据 SkinManager 获取当前皮肤 ID，动态选择蓝色或黄色图片组。

```
1 QString skinId = SkinManager::getInstance()->getCurrentSkinId();
2 bool isBlue = (skinId == "blue");
3 QVector<QPixmap> &normalImages = isBlue ? blueNormalImages : yellowNormalImages;
4 QVector<QPixmap> &upImages = isBlue ? blueUpImages : yellowUpImages;
5 QVector<QPixmap> &fallImages = isBlue ? blueFallImages : yellowFallImages;
```

4.2.4 碰撞检测

使用碰撞矩形进行 AABB 碰撞检测。

```
1 pipe->getPipeRect().intersects(bird->getCollisionRect())
```

4.2.5 分数计算

通过管道时分数加 1，使用 isScored() 标志避免重复计分。

```
1 if (!pipe->isScored() && birdX > pipe->getX() + Pipe::PIPE_WIDTH) {
2     counter->score();    // 计分
3     pipe->setScored(true); // 标记已计分
4 }
```

4.3 单元测试

针对每个功能设计单独测试用例，测试小鸟飞行、管道生成、移动管道、碰撞检测、计分等功能是否正常。

5 单元测试

5.1 测试案例定义

表 1: 测试案例

输入	输出	目的
按空格键	小鸟向上飞起, 速度为 14	小鸟飞行测试
松开空格	小鸟匀加速下落, 速度每帧 +2	小鸟下落测试
游戏开始	每隔一段时间生成新管道	管道生成测试
管道生成后	管道上下往复移动	移动管道测试
小鸟撞管道	游戏结束, 播放碰撞音效	碰撞检测测试
小鸟通过管道	分数加 1	计分测试
按 ←→ 键	皮肤在蓝/黄之间切换	皮肤切换测试

5.2 测试结果

测试结果全部通过。各功能模块经过多轮测试, 验证了游戏核心玩法的正确性。

6 收获

6.1 虚析构函数

本项目中, Pipe 基类继承自 QObject, QObject 具有虚析构函数, 因此通过基类指针删除子类对象时能正确调用子类的析构函数, 避免内存泄漏。

6.2 单元测试

更新功能代码时, 单元测试能够很快定位出问题的代码。当修改某个功能后, 运行单元测试即可快速验证修改是否正确, 大大提高了开发效率。

6.3 面向对象设计

通过本项目, 深入理解了面向对象编程的三大特性:

- 封装: 每个类职责单一, 数据私有
- 继承: Pipe → MovingPipe
- 多态: 虚函数实现运行时动态绑定

6.4 设计模式

- 单例模式: ScoreCounter、SkinManager 全局唯一实例
- 对象池模式: PipePool 复用管道对象, 避免频繁内存分配

6.5 Qt 框架

学习了 Qt 的核心组件:

- QWidget: 主窗口基类
- QPainter: 2D 图形绘制
- QTimer: 30 FPS 游戏循环
- 信号与槽: 组件间通信